

Deep Research: Avatrada Ui 9 Topic 003

Engineering Report: Client-Side High-Frequency Data Pipeline

TO: Lead Architect **FROM:** Autonomous Technical Researcher **DATE:** October 26, 2023 **SUBJECT:** Deep-Dive on SharedWorker-based Data Ingestion, Arbitration, and Throttling for the Avatrada UI

1.0 Executive Summary

This report provides a detailed engineering analysis of the proposed client-side data pipeline, as outlined in `avatrada_ui_9.pdf`. The core of this system is a `SharedWorker` designed to act as a centralized data hub for a high-performance trading application. It manages multiple WebSocket connections, arbitrates between redundant data sources, throttles high-frequency updates for smooth UI rendering, and implements resilience against UI thread freezes.

This analysis deconstructs the required mechanisms, provides concrete implementation strategies with code, and offers a critical analysis of potential failure modes and optimizations. The proposed architecture is robust, leveraging modern browser APIs to achieve high throughput and low latency while protecting the main UI thread from being overwhelmed.

2.0 SharedWorker Architecture for Connection Management

2.1 Technical Deconstruction

A `SharedWorker` is a specific type of Web Worker that can be accessed from multiple browsing contexts (windows, tabs, iframes) of the same origin. This

makes it the ideal architectural choice for our use case. Instead of each browser tab opening its own set of 5 WebSocket connections, a single `SharedWorker` establishes and maintains them once. This singleton pattern for data connections provides several key benefits:

- **Connection Pooling:** Drastically reduces network and server resource consumption by preventing redundant WebSocket handshakes and connections.
- **State Centralization:** All raw data, connection statuses, and arbitration logic reside in one isolated thread, ensuring data consistency across all UI tabs.
- **Resilience:** If a single UI tab crashes, the worker and its data connections persist, allowing other tabs to continue functioning and a new tab to connect instantly to the live data stream.

The worker's primary responsibilities are to manage the lifecycle of the 5 WebSocket connections (ThetaData, IBKR, Gemini, Tavily, Benzinga) and to manage communication with any connected UI tabs via `MessagePort` objects.

2.2 Implementation Strategy

The implementation involves two main parts: the worker script and the client-side connection logic.

`shared-worker.js` - The Worker Script

This script will initialize connections, manage ports, and broadcast data.

```
// shared-worker.js

// --- 1. State Management & Configuration ---
const ports = new Set(); // Stores MessagePorts for all connected tabs
const WEBSOCKET_CONFIG = {
  thetaData: { url: 'wss://source.thetaddata.net/stream', handler:
    handleThetaTick, instance: null },
  ibkr: { url: 'wss://source.ibkr.com/stream', handler: handleIbkrTick,
    instance: null },
  gemini: { url: 'wss://source.gemini.com/stream', handler:
```

```

handleGenericTick, instance: null },
    tavily: { url: 'wss://source.tavily.com/stream', handler:
handleGenericTick, instance: null },
    benzinga: { url: 'wss://source.benzinga.com/stream', handler:
handleGenericTick, instance: null },
};

// --- 2. Worker Connection Management ---
self.onconnect = (event) => {
    const port = event.ports[0];
    ports.add(port);
    console.log(`[Worker] New UI connection. Total clients: ${ports.size}`);

    port.onmessage = (e) => {
        // Handle commands from UI, e.g., subscriptions, or frame requests for
        throttling
        if (e.data.type === 'request_frame') {
            flushUpdateBuffer();
        }
    };

    port.start();

    // Note: There is no native 'onclose' for ports. See Section 5.0 for
    handling dead ports.
};

// --- 3. WebSocket Connection Logic ---
function connect(wsConfig) {
    const ws = new WebSocket(wsConfig.url);
    wsConfig.instance = ws;

    ws.onopen = () => console.log(`[Worker] WebSocket connected: ${wsConfig.url}`);

    ws.onmessage = (event) => {
        // Data arrives here. It's passed to a specific handler.
        const data = JSON.parse(event.data); // Or Protobuf decode
        wsConfig.handler(data);
    };
};

```

```

    ws.onerror = (err) => console.error(`[Worker] WebSocket error on $
{wsConfig.url}:`, err);

    ws.onclose = () => {
        console.log(`[Worker] WebSocket disconnected: ${wsConfig.url}.
Reconnecting in 5s...`);
        wsConfig.instance = null;
        setTimeout(() => connect(wsConfig), 5000); // Exponential backoff is
recommended
    };
}

// --- 4. Data Broadcasting ---
function broadcast(message) {
    // Stringify once before broadcasting for efficiency
    const payload = JSON.stringify(message);
    ports.forEach(port => {
        port.postMessage(payload);
    });
}

// --- 5. Initialization ---
function initializeConnections() {
    console.log('[Worker] Initializing all WebSocket connections...');
    for (const key in WEBSOCKET_CONFIG) {
        connect(WEBSOCKET_CONFIG[key]);
    }
}

// Dummy handlers for now - will be integrated with arbitration logic
function handleThetaTick(tick) { processArbitration('ThetaData', tick); }
function handleIbkrTick(tick) { processArbitration('IBKR', tick); }
function handleGenericTick(tick) { /* Process other streams */ }

initializeConnections();

// Arbitration and Buffer-Flush logic will be defined below...

```

2.3 Critical Analysis

- **Single Point of Failure (SPOF):** If the `SharedWorker` script encounters a fatal, unhandled exception, it will terminate. This will sever all data connections for *all* open tabs. Robust `try...catch` blocks around critical sections like message parsing and processing are essential.
 - **Port Lifecycle Management:** The `onconnect` event is well-defined, but there is no corresponding `ondisconnect`. If a tab is closed, its `MessagePort` remains in the `ports` set. This can lead to a memory leak and wasted cycles trying to `postMessage` to a dead port. A heartbeat/ping-pong mechanism is required to periodically prune dead ports (see Section 5.0).
 - **Browser Compatibility:** `SharedWorker` is widely supported in modern browsers, but not in all older versions or some mobile browsers. The application must have a graceful fallback, perhaps to a standard `Worker` per tab, if `SharedWorker` is not available.
-

3.0 Data Arbitration & Failover Algorithm

3.1 Technical Deconstruction

Data arbitration is the process of selecting the "best" data point from multiple redundant sources. In high-frequency trading, the most critical metric is recency. The goal is to create a single, unified stream of data that is always sourced from the provider with the lowest latency. The specified logic is a failover mechanism: Source A (ThetaData) is primary, but if its data becomes stale relative to Source B (IBKR) by more than 300ms, the system must switch to Source B and notify the UI of potential data degradation.

3.2 Implementation Strategy

This logic will reside entirely within the `SharedWorker`. It requires maintaining state about the current primary source and the last seen timestamps from each.

```
// shared-worker.js (continued)
```

```

// --- Arbitration State ---
let primarySource = 'ThetaData'; // Preferred source
let lastTimestamp = {
  ThetaData: 0,
  IBKR: 0
};
const FAILOVER_THRESHOLD_MS = 300;

/**
 * The core arbitration logic. Receives ticks from all sources
 * and decides which one to pass to the buffer.
 * @param {string} sourceName - 'ThetaData' or 'IBKR'
 * @param {object} tick - The tick data, must include 'exchange_timestamp'
 */
function processArbitration(sourceName, tick) {
  if (!tick.exchange_timestamp) return; // Ignore ticks without a valid
  timestamp

  // Update the last seen timestamp for the incoming source
  lastTimestamp[sourceName] = tick.exchange_timestamp;

  // --- Failover Logic ---
  if (primarySource === 'ThetaData' && sourceName === 'IBKR') {
    // We received a tick from the backup source. Check if the primary is
    lagging.
    if (lastTimestamp.ThetaData < (tick.exchange_timestamp -
    FAILOVER_THRESHOLD_MS)) {
      console.warn(`[Arbitrator] ThetaData lag detected (>300ms). Failing
      over to IBKR.`);
      primarySource = 'IBKR';
      broadcast({ type: 'system_event', event: 'Data Degraded',
      newSource: 'IBKR' });
    }
  } else if (primarySource === 'IBKR' && sourceName === 'ThetaData') {
    // We received a tick from the original primary. Check if it has
    recovered.
    // For simplicity, we switch back as soon as a fresh tick arrives.
    // A more robust system might require it to be consistently faster.
    if (tick.exchange_timestamp > lastTimestamp.IBKR) {
      console.info(`[Arbitrator] ThetaData has recovered. Switching back

```

```

as primary.`);
    primarySource = 'ThetaData';
    broadcast({ type: 'system_event', event: 'Data Restored',
newSource: 'ThetaData' });
    }
}

// --- Stream Selection ---
// Only forward the tick to the buffer if it comes from the current primary
source.
if (sourceName === primarySource) {
    // This tick is from our trusted source, add it to the UI update buffer.
    addToUpdateBuffer(tick);
}
}

```

3.3 Critical Analysis

- **Clock Skew:** The entire algorithm relies on `exchange_timestamp`. This assumes that the clocks at the ThetaData and IBKR exchanges are perfectly synchronized. Any significant clock skew between the two exchanges could trigger a false failover. While exchange clocks are typically synchronized via NTP, this is an external dependency we cannot control.
- **"Flapping":** If the latency difference between the two sources hovers around 300ms, the primary source could switch back and forth rapidly. This "flapping" can be disruptive. To mitigate this, a hysteresis mechanism should be implemented:
 1. **Failover Cooldown:** After failing over to IBKR, do not switch back to ThetaData for a minimum period (e.g., 10 seconds), even if it recovers.
 2. **Stricter Recovery Condition:** To switch back to ThetaData, require its timestamp to be consistently ahead of IBKR's for several consecutive ticks.
- **Heartbeat Requirement:** The current logic only triggers on an incoming tick from the backup source. If the primary source (ThetaData) dies completely and stops sending ticks, and the backup (IBKR) also has a lull in data, the system won't know the primary is dead. This is where the

"Heartbeat Analysis" from the source document is critical. Each source should send a heartbeat message every ~250ms. The worker must run a `setInterval` check to ensure these heartbeats are received. If a heartbeat from the primary is missed, it should trigger an immediate failover.

4.0 Buffer-Flush Mechanism using `requestAnimationFrame`

4.1 Technical Deconstruction

The data sources can produce thousands of updates per second, while a browser UI can only repaint at the monitor's refresh rate (typically 60Hz, or ~16.7ms per frame). Sending every single tick to the main thread would queue up an overwhelming number of tasks, causing the UI to become unresponsive and eventually crash.

The solution is to batch and aggregate updates in the worker. The worker will maintain the "latest state" of the data. The UI thread will then use `requestAnimationFrame` (rAF) to "pull" a single, consolidated update from the worker just before it needs to render a new frame. This perfectly synchronizes data updates with the browser's rendering cycle.

4.2 Implementation Strategy

We will use a `Map` in the worker to store the aggregated data. The key will be the trading symbol, and the value will be an object containing the latest price and accumulated volume delta.

```
// shared-worker.js (continued)

// --- Buffer-Flush State ---
// Map<symbol, { lastPrice: number, volumeDelta: number, lastUpdate: number }>
let updateBuffer = new Map();

/**
 * Called by the arbitrator for every tick from the primary source.
```

```

    * Aggregates data into the buffer.
    * @param {object} tick - { symbol: 'SPY', price: 450.10, volume: 100,
exchange_timestamp: ... }
    */
function addToUpdateBuffer(tick) {
    if (updateBuffer.has(tick.symbol)) {
        // Aggregate existing entry
        const existing = updateBuffer.get(tick.symbol);
        existing.lastPrice = tick.price;
        existing.volumeDelta += tick.volume; // Assuming tick.volume is the
trade size
        existing.lastUpdate = tick.exchange_timestamp;
    } else {
        // Create new entry
        updateBuffer.set(tick.symbol, {
            lastPrice: tick.price,
            volumeDelta: tick.volume,
            lastUpdate: tick.exchange_timestamp
        });
    }
}

/**
 * Called when the UI thread requests a new frame of data.
 * Packages the aggregated data and sends it.
 */
function flushUpdateBuffer() {
    if (updateBuffer.size === 0) {
        return; // Nothing to send
    }

    // Convert Map to an array of objects for serialization
    const payload = Array.from(updateBuffer.entries()).map(([symbol, data]) =>
({
        symbol,
        ...data
    }));

    broadcast({
        type: 'market_update',

```

```

        ticks: payload
    });

    // Clear the buffer for the next frame
    updateBuffer.clear();
}

```

main.js - The UI Thread Logic

The client-side code is responsible for driving the flush cycle.

```

// main.js - In your UI code

const sharedWorker = new SharedWorker('shared-worker.js');
const workerPort = sharedWorker.port;

workerPort.onmessage = (event) => {
    const data = JSON.parse(event.data);
    if (data.type === 'market_update') {
        // This function will be called ~60 times per second
        renderUI(data.ticks);
    } else if (data.type === 'system_event') {
        showNotification(data.event);
    }
};

// The rAF loop that drives the data flushing from the worker
function animationLoop() {
    // Tell the worker we are ready for the next batch of data
    workerPort.postMessage({ type: 'request_frame' });

    // Schedule the next request
    requestAnimationFrame(animationLoop);
}

// Start the loop
requestAnimationFrame(animationLoop);

```

4.3 Critical Analysis

- **Throttling in Background Tabs:** A major advantage of this pull-based `rAF` approach is that browsers automatically throttle `requestAnimationFrame` for background tabs (typically to $\sim 1\text{Hz}$). This means the worker will naturally send fewer updates to inactive tabs, saving significant CPU resources on both the worker and main threads.
 - **Data Loss vs. Aggregation:** This is a "lossy" process by design. We are intentionally discarding intermediate price points between frames in favor of the latest price and an aggregated volume. This is the correct trade-off for UI visualization. For analytics or charting that requires every tick, a separate, non-throttled data path would be needed.
 - **Ring Buffers:** The source document mentions Ring Buffers. While our `Map` handles the aggregation, a Ring Buffer could be used for a different purpose: storing a fixed-size history of *raw* incoming ticks before aggregation. This is excellent for avoiding garbage collection (GC) pauses, as you continuously reuse a pre-allocated array. However, for the described aggregation logic (last price/volume delta), a `Map` is more semantically direct and its performance is generally excellent. A hybrid approach could use a Ring Buffer for the raw ingress, with the aggregation `Map` being populated from it.
-

5.0 Backpressure Handling and Memory Leak Prevention

5.1 Technical Deconstruction

Backpressure occurs when the data producer (worker) creates messages faster than the consumer (UI thread) can process them. If the UI thread freezes (e.g., due to a long-running script, heavy GC, or debugging breakpoint), the browser's internal message queue for `postMessage` can grow without bound, leading to a massive memory leak and eventual crash. The worker must be able to detect a non-responsive UI and stop sending it data.

5.2 Implementation Strategy

A ping-pong heartbeat mechanism is a simple and effective way to detect a frozen UI thread. The worker periodically "pings" each connected client, which must "pong" back within a timeout.

```
// shared-worker.js (continued)

// --- Backpressure Management ---
const PING_INTERVAL_MS = 2000;
const PONG_TIMEOUT_MS = 1000;

// Use a Map to track the health of each port
// Map<port, { lastPong: number, isAlive: boolean }>
const portHealth = new Map();

// Modify the onconnect handler
self.onconnect = (event) => {
  const port = event.ports[0];
  ports.add(port);
  portHealth.set(port, { lastPong: Date.now(), isAlive: true });
  console.log(`[Worker] New UI connection. Total clients: ${ports.size}`);

  port.onmessage = (e) => {
    if (e.data.type === 'request_frame') {
      flushUpdateBuffer();
    } else if (e.data.type === 'pong') {
      // UI is responsive, update its health status
      if (portHealth.has(port)) {
        portHealth.get(port).lastPong = Date.now();
        portHealth.get(port).isAlive = true;
      }
    }
  };
  port.start();
};

// Health check loop
setInterval(() => {
```

```

const now = Date.now();

// Check for dead ports
portHealth.forEach((health, port) => {
  if (now - health.lastPong > PING_INTERVAL_MS + PONG_TIMEOUT_MS) {
    if (health.isAlive) {
      console.warn('[Worker] UI port timed out. Assuming dead or
frozen.');
```

```

      health.isAlive = false;
    }
  }
});

// Prune dead ports from the main set
ports.forEach(port => {
  if (portHealth.has(port) && !portHealth.get(port).isAlive) {
    console.log('[Worker] Pruning dead port.');
```

```

    ports.delete(port);
    portHealth.delete(port);
  }
});

// Send a new ping to all living ports
ports.forEach(port => {
  port.postMessage(JSON.stringify({ type: 'ping' }));
});

}, PING_INTERVAL_MS);

// Modify the broadcast function to respect port health
function broadcast(message) {
  const payload = JSON.stringify(message);
  ports.forEach(port => {
    // This check is now redundant if we prune dead ports, but serves as a
    good safeguard.
    if (portHealth.get(port)?.isAlive) {
      port.postMessage(payload);
    }
  });
}

```

main.js - UI Thread Pong Response

The UI must listen for pings and respond immediately.

```
// main.js - In your UI code

workerPort.onmessage = (event) => {
  const data = JSON.parse(event.data);
  if (data.type === 'ping') {
    workerPort.postMessage({ type: 'pong' });
    return; // Respond immediately
  }
  // ... other message handlers
};
```

5.3 Critical Analysis

- **Timeout Tuning:** The `PING_INTERVAL_MS` and `PONG_TIMEOUT_MS` values are critical. If the timeout is too short, a temporarily busy but healthy UI thread might be incorrectly flagged as dead. If it's too long, the message queue can grow excessively before the worker stops sending data. These values should be tuned based on expected UI load.
- **Recovery:** When a frozen UI unfreezes, it will start responding to pings again. However, in our implementation, its port has already been removed. The UI would need to detect that it's no longer receiving data (e.g., a timeout on its end) and re-establish a connection to the `SharedWorker`, which would create a new `MessagePort`.
- **Alternative to Pruning:** Instead of completely removing the port, the worker could simply mark it as `isAlive: false` and stop sending high-frequency `market_update` messages. It could continue sending low-frequency pings. If a `pong` is eventually received, it could set `isAlive: true` and resume the data stream. This provides a more seamless recovery for temporarily frozen UIs.